

4. Production Database, Storage & Mobile OTP

Target Goal: Establish persistent farmer accounts, geospatial field boundaries, compressed cloud photo storage, and seamless phone OTP login.

Step 1: Production Database Schema (PostgreSQL + PostGIS)

Run these DDL statements in Supabase, Neon, or your PostgreSQL instance:

```
-- Enable PostGIS for field boundary coordinates
CREATE EXTENSION IF NOT EXISTS postgis;

-- 1. Farmers Table
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  mobile_number VARCHAR(15) UNIQUE NOT NULL,
  full_name VARCHAR(100) NOT NULL,
  village VARCHAR(100),
  district VARCHAR(100),
  state VARCHAR(100),
  preferred_language VARCHAR(10) DEFAULT 'hi',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 2. Farms & Fields
CREATE TABLE fields (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  field_name VARCHAR(100) NOT NULL,
  crop VARCHAR(50) NOT NULL,
  variety VARCHAR(100),
  area_acres NUMERIC(5,2) NOT NULL,
  sowing_date DATE,
  boundary_geom GEOMETRY(POLYGON, 4326), -- GPS Field Polygon
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 3. Scans & AI Diagnoses
CREATE TABLE crop_scans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  field_id UUID REFERENCES fields(id) ON DELETE CASCADE,
  image_url TEXT NOT NULL,
  image_phash VARCHAR(64) NOT NULL,
  severity_percentage NUMERIC(5,2) NOT NULL,
  affected_area_percentage NUMERIC(5,2) NOT NULL,
  detected_diseases JSONB NOT NULL,
  weather_snapshot JSONB,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 4. Index for fast duplicate image lookups
CREATE INDEX idx_scans_phash ON crop_scans(image_phash);
```

Step 2: Passwordless Mobile OTP Login with Firebase Auth

Small farmers do not use passwords. Use Firebase Phone Auth for instant SMS OTP verification:

```
// src/services/phoneAuth.ts
import { getAuth, RecaptchaVerifier, signInWithPhoneNumber } from 'firebase/auth';

// 1. Initialize invisible reCAPTCHA
export function setupRecaptcha(buttonId: string) {
  const auth = getAuth();
  return new RecaptchaVerifier(auth, buttonId, {
    size: 'invisible'
  });
}

// 2. Send SMS OTP
export async function sendOtpToMobile(phoneNumber: string, appVerifier: any) {
  const auth = getAuth();
  const confirmationResult = await signInWithPhoneNumber(auth, `+91${phoneNumber}`, appVerifier);
  return confirmationResult; // Call confirmationResult.confirm(otpCode) on user input
}
```

Step 3: Client-Side WebP Compression & Cloudflare R2 Upload

Never upload raw 5MB camera photos over rural 3G networks. Compress images to lightweight WebP (<200KB) in the browser before sending:

```
// Compresses canvas image to WebP blob
export async function compressLeafImage(file: File): Promise<Blob> {
  return new Promise((resolve) => {
    const img = new Image();
    img.src = URL.createObjectURL(file);
    img.onload = () => {
      const canvas = document.createElement('canvas');
      const MAX_WIDTH = 1200;
      const scale = MAX_WIDTH / img.width;
      canvas.width = MAX_WIDTH;
      canvas.height = img.height * scale;

      const ctx = canvas.getContext('2d')!;
      ctx.drawImage(img, 0, 0, canvas.width, canvas.height);

      canvas.toBlob((blob) => resolve(blob!), 'image/webp', 0.82);
    };
  });
}
```